

LSH Near Duplicate Document Detection

Nicola Castelletti, Pietro Stangherlin

Indice

- Background
- Obiettivi
- Metodi utilizzati
- Esperimenti
- Conclusioni

Background

- Problema: in grandi raccolte di documenti (in particolare testuali) è possibile che vi siano dei documenti duplicati o quasi uguali; è inoltre possibile che tali documenti abbiano identificativi diversi o comunque non direttamente riconducibili tra loro.
- Esempi
 - medesimi documenti scannerizzati più volte da entità diverse
 - un'organizzazione che mantiene versioni diverse, ovvero con lievi modifiche, di uno stesso documento

Obiettivi

Implementare un metodo basato su MinHash e LSH per identificare i documenti duplicati e quasi-duplicati.

Valutarne l'efficacia e ed effettuare considerazioni sull'efficienza.

Metodi utilizzati

- Creazione della collezione sperimentale
- Shingling
- MinHash
- LSH

Creazione della collezione sperimentale

- 1 Si assume di avere una collezione iniziale senza duplicati e quasi – duplicati. Ogni documento presenta un codice identificativo (id).
- 2 Si seleziona (ad esempio tramite un generatore di numeri casuali) un sottoinsieme di documenti da cui generare i duplicati e i quasi duplicati.
- 3 Quasi duplicati: sostituzione di alcuni caratteri con altri (casuali), per simulare un risultato di un programma di riconoscimento ottico di testo. Il parametro relativo alla frequenza di sostituzione è la percentuale sul numero totale di caratteri nel testo
- 4 A ogni quasi-duplicato si assegna un id univoco (id2) e l'id del documento da cui è stato generato (id3). Viene creata una nuova collezione con i quasi-duplicati e i documenti originali (per questi ultimi il campo l'id3 è posto uguale a None)

Id	text
1	hello world
2	Bob, Alice and Ulm
3	nosce te ipsum



id2	id3	text
1	None	hello world
2	None	Bob, Alice and Ulm
3	None	nosce te ipsum
4	3	n@sCe t3 ipsuN

MinHash

Dimensione delle shingles: $w = 9$ **leskovec2014mining** (parametro fissato, seguendo le indicazioni sul libro di testo)

Salvataggio signatures

- scrittura su disco (database sql) (scelta impiegata)
- memoria centrale (es. btree) (non considerata, possibile per piccole collezioni)

Stima della memoria occupata dalle signatures y :

$$y = d \times p \times n$$

Con

- d : bit occupati da ciascun elemento della signature
- p = n° di permutazioni
- s = n° di documenti nella collezione

Esempio: con 100 permutazioni, interi a 32 bit e 10^6 documenti: $ms = 400$ megabyte.

SignatureHash

Per approssimare ciascuna permutazione in ogni elemento delle signature si impiegano delle funzioni hash. Come descritto in **BroderMin-wise**, alla base di molti schemi di hashing, tra cui quello considerato, vi è che ogni funzione hash $h : U \rightarrow [m]$ sia casuale. Tale assunzione non può ovviamente essere soddisfatta e si ricorre dunque a delle approssimazioni. Si dice che una famiglia di funzioni hash $H = \{h | h : U \rightarrow [m]\}$ è (debolmente) universale se $\forall x_1 \neq x_2 \in U$ e estraendo h uniformemente da H si ha $\Pr\{h(x_1) = h(x_2)\} \leq \frac{1}{m}$. Una semplice implementazione di una famiglia di funzioni hash universale è $h(x) = [(a * x + b) \bmod p] \bmod m$, con

- a, b interi estratti casualmente
- p numero primo (maggiore di m)

LSH

Implementazioni

- in memoria di massa (non considerata)
- in memoria centrale: ogni bucket è implementato come una linked list
 - Lista: ogni banda ha $P * N$ bucket con N numero di documenti e p fattore di scala > 1 , il tempo di accesso ad ogni bucket è $O(1)$, al costo di memoria occupata dai bucket vuoti.
 - Btree: non sono presenti bucket vuoti, il tempo di accesso a ogni bucket è $O(\log_2[N])$
- in memoria centrale in un cluster (ad esempio via MapReduce)

In tutti i casi è conveniente salvare gli indirizzi dei bucket con almeno due elementi (evita inutili controlli successivi).

Per ogni banda, considerata una signature $v = (v_1, v_2, \dots, v_d)^\top$ si impiega la funzione hash $h(x) = \left(\sum_{j=1}^d v_j a_j \right)$ modulo $N * P$ con $a = (a_1, a_2, \dots, a_d)^\top$ vettore di numeri (pseudo)casuali.

Misure di similarità

- numero di bucket condivisi tra due documenti: ottenuta da LSH
- similarità delle signature: l'accesso ai record del database per ogni confronto è molto lento, si è quindi ricorsi ad un caching delle signatures: per tutti i documenti presenti in almeno una coppia si popola un dizionario di signature. Popolare il dizionario ha un costo di $O(N)$ (lettura dei record del database), ma ogni confronto richiederà $O(1)$ per leggere le signature.

Esperimenti

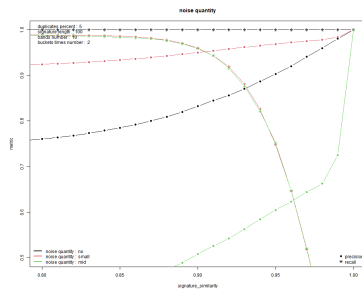
- Prima esecuzione della procedura sulla collezione originale (senza semi-duplicati artificiali) per tarare i parametri e valutare il massimo grado di similarità presente tra documenti della collezione originale. Rimozione dei documenti con similarità alta per migliorare le valutazioni sui dati semi-artificiali.
- Esecuzioni successive con diversi livelli di modifica dei documenti semi-duplicati artificiali
- Problemi:
 - lo spazio dei parametri è vasto
 - gli articoli non forniscono molte indicazioni
 - ottenere i risultati di una singola configurazione è costoso

Parametri degli esperimenti

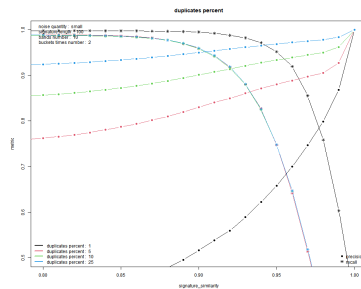
- Rumore nei semi-duplicati: nessuno (0%), “piccolo” (2%), “medio” (5%)
- Percentuale di duplicati relativo alla dimensione della collezione: 1%, 5%, 10%, 25%.
- Lunghezza signature: 100, 200
- Numero di bande: 10, 20
- Numero di bucket relativo alla dimensione della collezione: 2, 5, 10

Combinazioni totali per collezione: 192.

Risultati Low

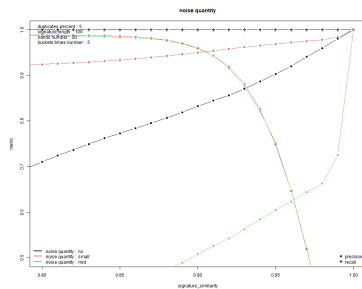


(a) low 1

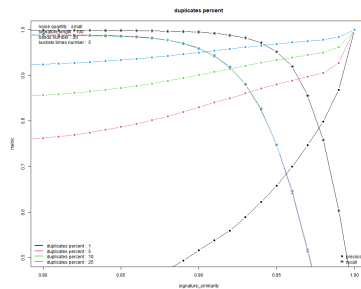


(b) low 2

Risultati Mid

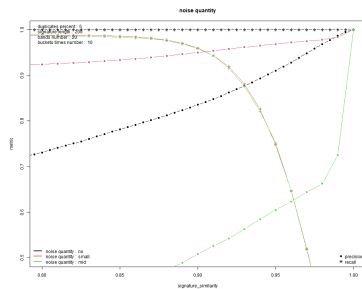


(a) mid 1

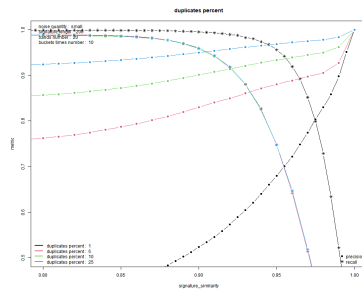


(b) mid 2

Risultati High

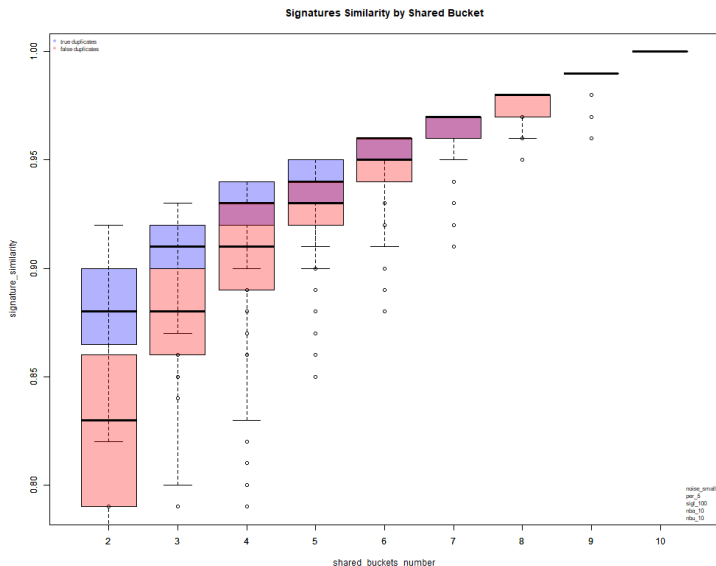


(a) high 1



(b) high 2

Risultati Signature Similarity vs Shared Bucket



Conclusioni

Difficoltà emerse:

- La creazione della collezione per la valutazione dell'effetto dei parametri ha richiesto delle scelte arbitrarie.
- Lo spazio dei parametri è vasto.
- Python non permette una accurata gestione della memoria (a differenza ad esempio di C), questo può essere sia un pro che un contro, nel nostro caso ciò ha reso difficilmente interpretabile l'impiego di memoria tramite profiling.

Cosa si è imparato:

- Documentare le funzioni può richiedere più tempo della scrittura del codice stesso
- Scalare il metodo senza sacrificare eccessiva velocità richiede investimenti in termini di memoria centrale (eventualmente noleggiando dei server)

Sviluppi futuri

- Valutare le differenze con diverse tipologie di collezioni